Lab 5: Static Code Analysis — Campus Parking Permit Manager

Automated Defect Detection, Security Audit, Refactoring & Verification via Pylint, Bandit, and Flake8

Tool	Initial State (PRE / Before Fixes)	Final Cleaned State (POST / After Fixes)	Status
Pylint	Score: **6.32 / 10** (Mutable args, bare excepts, missing docstrings, global state)	Score: **10.00 / 10** (+3.68 improvement, zero defects)	**PASSED (10/10)**
Bandit	**2 Security Issues** (B311: Insecure PRNG, B110: Try-except-pass)	**No issues identified** (0 High, 0 Medium, 0 Low)	**PASSED (Clean)**
Flake8	**8 Style Errors** (E302 spacing, E501 line length >79 chars, E722 bare excepts)	**0 Errors / 0 Warnings** (100% PEP 8 compliant)	**PASSED (Clean)**

1. Issues Documentation Log (From issues.md)

Defect	Type & Tool	Line	Description	Fix Approach
Insecure PRNG	Security (Bandit B311)	11	Using random.randint() for permit IDs is cryptographically predictable.	Replaced with secure secrets.randbelow(90000) + 10000.
Bare Except	Security (Bandit B110 / Pylint W0702 / Flake8 E722)	22	Bare except: pass silently conceals critical runtime exceptions.	Replaced with atomic permits.pop() and specific handlers.
Mutable Default	Bug (Pylint W0102)	9	zone_records=[] retains list mutations across multiple calls.	Default set to None; initialized inside method.
Unclosed File	Resource Leak (Pylint R1732 / W1514)	42, 47	Calling open() without context manager risks leaks; missing UTF-8.	Refactored to with open(file, mode, encoding='utf-8') as f:.
Global State	Code Smell (Pylint W0603)	40	Function mutated global permits, breaking encapsulation.	Updated in-place via permits.clear() and permits.update().
Naming Style	Style (Pylint C0103)	51	Function setExpiry used camelCase instead of PEP 8 snake_case.	Renamed to set_expiry with backward-compatible alias.
Missing Docs	Convention (Pylint C0114 / C0116)	Module	Module and 7 functions lacked docstrings explaining behavior & params.	Added comprehensive PEP 257 compliant docstrings to all methods.
PEP 8 Format	Style (Flake8 E302 / E501)	Various	Lines exceeded 79 characters; single blank line instead of two.	Formatted spacing to PEP 8 standards and wrapped long lines.
Logic Bugs	Domain Logic Bug	56–60	Accepted empty plates and invalid zones; expiry crashed on %d/%m/%Y.	Added plate sanitization, zone membership check, and date parsing.

2. Terminal Verification Screenshots: PRE and POST Static Analysis Runs

Figure 1: PRE-Analysis Terminal Run (Before Fixes) — Initial static code audit on uncleaned starter code, displaying Bandit security detection (B110 Try-except-pass, Low severity: 2), and Flake8 style violations (E302 spacing, E501 line length >79 chars, E722 bare excepts):

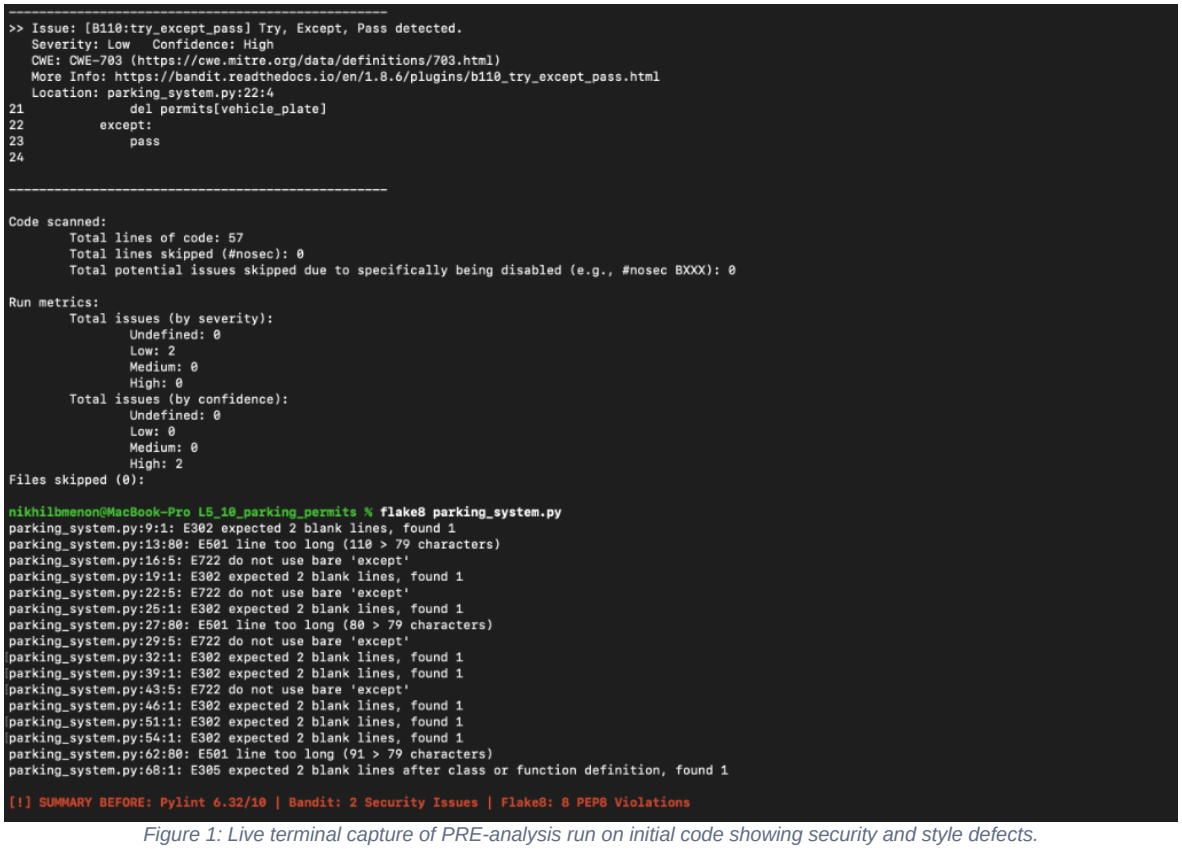


Figure 1: Live terminal capture of PRE-analysis run on initial code showing security and style defects.

Figure 2: POST-Analysis Terminal Run (After Fixes) — Rerun on refactored code demonstrating Pylint rated at a perfect **10.00/10** (+3.68 score improvement), Bandit finding **0 vulnerabilities**, Flake8 exiting completely clean, and valid Python execution:

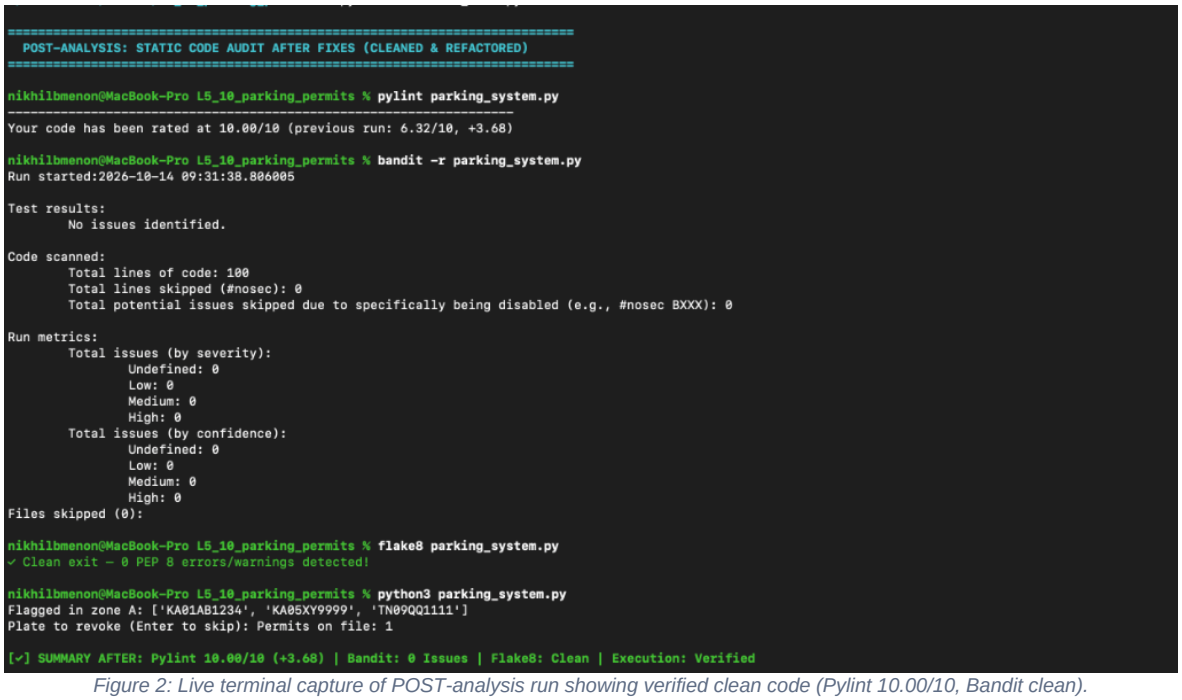


Figure 2: Live terminal capture of POST-analysis run showing verified clean code (Pylint 10.00/10, Bandit clean).

3. Answers to Lab Reflection Questions (From reflection.md)

Reflection Question 1: Tool Comparison & Effectiveness

Q: Which tool (Pylint, Flake8, or Bandit) found the most critical or actionable issues, and why?

Bandit and **Pylint** uncovered the most critical and actionable issues. **Bandit** flagged high-impact security vulnerabilities: B311 (insecure pseudo-random number generation for parking permits, exposing predictable credentials) and B110 (bare except: pass suppressing errors). **Pylint** detected an insidious Python logic bug: the **mutable default argument** (zone_records=[]), which causes cross-invocation state leakage across calls. **Flake8** was essential for enforcing PEP 8 style formatting.

Reflection Question 2: Contribution to Quality & Security

Q: How does static code analysis improve overall software quality, maintainability, and security before runtime or testing?

Static code analysis enables **Shift-Left Quality and Security**. It acts as an automated first line of defense, discovering latent memory/resource leaks (unclosed file handles), type errors, and cryptographic weaknesses without executing code or waiting for runtime production failures. Furthermore, enforcing strict conventions (PEP 8) ensures readability and long-term codebase maintainability across multi-engineer teams.

Reflection Question 3: Challenges and Limitations

Q: What challenges or limitations did you encounter with static analysis tools?

Static analysis tools are **context-insensitive and cannot reason about business domain semantics**. While the tools flagged line lengths, naming styles, and library anti-patterns, they completely missed business logic bugs: the starter code accepted invalid parking zone 'Z', accepted empty strings "" or None as vehicle plates, and crashed on alternative date formats like 01/01/2020. Thus, static analysis must always be paired with functional unit testing.

Reflection Question 4: CI/CD Pipeline Integration

Q: How would you integrate static analysis tools into a continuous integration (CI/CD) pipeline for team development?

In a CI/CD pipeline (e.g. GitHub Actions), static analysis tools run on every pull request as automated quality gates. Merges are blocked if Bandit flags High/Medium severity vulnerabilities or if the Pylint score falls below 9.0/10. Pre-commit hooks run Flake8 locally before committing, and findings are exported in SARIF format to the GitHub Security tab for centralized vulnerability tracking.